

Resources ▼

cat

Concatenate arrays

Syntax

```
C = cat(dim,A,B)
C = cat(dim,A1,A2,...,An)
```

Description

`C = cat(dim,A,B)` concatenates `B` to the end of `A` along dimension `dim` when `A` and `B` have compatible sizes (the lengths of the dimensions match except for the operating dimension `dim`). [example](#)

`C = cat(dim,A1,A2,...,An)` concatenates `A1`, `A2`, ..., `An` along dimension `dim`. [example](#)

You can use the square bracket operator `[]` to concatenate or append arrays. For example, `[A,B]` and `[A B]` concatenates arrays `A` and `B` horizontally, and `[A; B]` concatenates them vertically.

Examples

[collapse all](#)

Two Matrices

Concatenate two matrices vertically, then horizontally.

Create two matrices, and vertically append the second matrix to the first.

[Open in MATLAB Online](#)[Copy Command](#)

```
A = ones(3)
```

[Get](#) ▼

`A = 3×3`

```
1     1     1
1     1     1
1     1     1
```

```
B = zeros(3)
```

[Get](#) ▼

`B = 3×3`

```
0     0     0
0     0     0
0     0     0
```

```
C1 = cat(1,A,B)
```

[Get](#) ▼

C1 = 6×3

1	1	1
1	1	1
1	1	1
0	0	0
0	0	0
0	0	0

Now, horizontally append the second matrix to the first.

C2 = cat(2,A,B)

 Get ▾

C2 = 3×6

1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0

3-D Arrays

Create two 3-D arrays and concatenate them along the third dimension. The lengths of the first and second dimensions in the resulting array match the corresponding lengths in the input arrays, while the third dimension expands.

[Open in MATLAB Online](#)

 [Copy Command](#)

```
A = rand(2,3,4);
B = rand(2,3,5);
C = cat(3,A,B);
szC = size(C)
```

 Get ▾

szC = 1×3

2	3	9
---	---	---

Expand Tables

Create a table and add a row using a cell array.

[Open in MATLAB Online](#)

 [Copy Command](#)

```
LastName = {'Sanchez';'Johnson';'Li';'Diaz'};
Age = [38;43;38;40];
```

 Get ▾

```
T1 = table(LastName, Age)
```

T1=4×2 table

LastName	Age
{ 'Sanchez' }	38
{ 'Johnson' }	43
{ 'Li' }	38
{ 'Diaz' }	40

```
Trow = { 'Brown', 49 };
T2 = cat(1, T1, Trow)
```

[Get](#)

T2=5×2 table

LastName	Age
{ 'Sanchez' }	38
{ 'Johnson' }	43
{ 'Li' }	38
{ 'Diaz' }	40
{ 'Brown' }	49

Dates with Different Types

Concatenate a date character vector, a string date, and a datetime into a single column of dates. The result is a datetime vector.

[Open in MATLAB Online](#)
[Copy Command](#)

```
chardate = '2016-03-24';
strdate = "2016-04-19";
t = datetime('2016-05-10', 'InputFormat', 'yyyy-MM-dd');
C = cat(1, chardate, strdate, t)
```

[Get](#)

C = 3×1 datetime

```
24-Mar-2016
19-Apr-2016
10-May-2016
```

Matrices in a Cell Array

Create a cell array containing two matrices, and concatenate the matrices both vertically and horizontally.

[Open in MATLAB Online](#)

[Copy Command](#)

```
M1 = [1 2; 3 4];  
M2 = [5 6; 7 8];  
A1 = {M1,M2};  
Cvert = cat(1,A1{:})
```

[Get](#)

Cvert = 4×2

1	2
3	4
5	6
7	8

```
Chorz = cat(2,A1{:})
```

[Get](#)

Chorz = 2×4

1	2	5	6
3	4	7	8

Input Arguments

[collapse all](#)

dim — Dimension to operate along
positive integer scalar

Dimension to operate along, specified as a positive integer scalar. For example, if A and B are both 2-by-2 matrices, then `cat(1,A,B)` concatenates vertically creating a 4-by-2 matrix. `cat(2,A,B)` concatenates horizontally creating a 2-by-4 matrix.

`dim` must be either 1 or 2 for table or timetable input.

A — First input
scalar | vector | matrix | multidimensional array | table | timetable

First input, specified as a scalar, vector, matrix, multidimensional array, table, or timetable.

B — Second input
scalar | vector | matrix | multidimensional array | table | timetable

Second input, specified as a scalar, vector, matrix, multidimensional array, table, or timetable.

- The elements of B are concatenated to the end of the first input along the operating dimension. The sizes of the input arguments must be compatible. For example, if the first input is a matrix of size 3-by-2, then B must have 2 columns to concatenate vertically, and 3 rows to concatenate horizontally.

- When concatenating horizontally, all table inputs must have unique variable names. When present, row names must be identical, except for order. Similarly, all timetable inputs must have unique variable names. Row times must be identical, except for order.
- You can concatenate valid combinations of different types. For more information, see [Valid Combinations of Text and Numeric Datatypes](#).

A1,A2,...,An — List of inputs comma-separated list

List of inputs, specified as a comma-separated list of arrays to concatenate in the order they are specified.

- The inputs must have compatible sizes. For example, if A1 is a row vector of length m , then the remaining inputs must each have m columns to concatenate vertically.
- When concatenating horizontally, all table inputs must have unique variable names. When present, row names must be identical, except for order. Similarly, all timetable inputs must have unique variable names. Row times must be identical, except for order.
- You can concatenate valid combinations of different types. For more information, see [Valid Combinations of Text and Numeric Datatypes](#).

Tips

- To construct text by horizontally concatenating strings, character vectors, or cell arrays of character vectors, use the [append](#) function.
- To construct a single piece of delimited text from a cell array of character vectors or a string array, use the [join](#) function.

Algorithms

When concatenating an empty array to a nonempty array, `cat` omits the empty array in the output. For example, `cat(2,[1 2],[ ])` returns the row vector `[1 2]`.

If all input arguments are empty and have compatible sizes, then `cat` returns an empty array whose size is equal to the output size as when the inputs are nonempty. For example, `cat(2,zeros(0,1),zeros(0,2))` returns a 0-by-3 empty array.

Extended Capabilities

[collapse all](#)

Tall Arrays

Calculate with arrays that have more rows than fit in memory.

The `cat` function supports tall arrays with the following usage notes and limitations:

- Vertical concatenation of character arrays is not supported.
- Concatenation of ordinal categorical arrays is not supported.
- Concatenation in any dimension other than 1 requires all input arguments to be tall arrays.

For more information, see [Tall Arrays](#).

C/C++ Code Generation

Generate C and C++ code using MATLAB® Coder™.

Usage notes and limitations:

- The `dim` argument must be a constant at code generation time.

GPU Code Generation

Generate CUDA® code for NVIDIA® GPUs using GPU Coder™.

Usage notes and limitations:

- The `dim` argument must be a constant at code generation time.

Thread-Based Environment

Run code in the background using MATLAB® `backgroundPool` or accelerate code with Parallel Computing Toolbox™ `ThreadPool`.

The `cat` function fully supports thread-based environments. For more information, see [Run MATLAB Functions in Thread-Based Environment](#).

GPU Arrays

Accelerate code by running on a graphics processing unit (GPU) using Parallel Computing Toolbox™.

The `cat` function supports GPU array input with these usage notes and limitations:

- Sparse matrices are not supported.

For more information, see [Run MATLAB Functions on a GPU](#) (Parallel Computing Toolbox).

Distributed Arrays

Partition large arrays across the combined memory of your cluster using Parallel Computing Toolbox™.

The `cat` function fully supports distributed arrays. For more information, see [Run MATLAB Functions with Distributed Arrays](#) (Parallel Computing Toolbox).

Version History

Introduced before R2006a

[expand all](#)

R2025a: Improved GPU code generation support

See Also

[vertcat](#) | [horzcat](#) | [append](#) | [join](#)

Topics

[Creating, Concatenating, and Expanding Matrices](#)

[Valid Combinations of Text and Numeric Datatypes](#)

[Combine Categorical Arrays](#)

[Concatenating Objects of Different Classes](#)

[Concatenation Methods](#)

YOU MIGHT ALSO BE INTERESTED IN

[Converting Deep Learning Models Between PyTorch, TensorFlow, and MATLAB](#)

Import and export AI models between PyTorch®, TensorFlow™, and MATLAB®.

[Learn more](#)